

On-Device AI 실습

KV Cache Pruning & Speculative Decoding

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

1. **KV Cache 메모리 계산**
 - Model의 Config로부터 캐시 바이트 수 계산, weight와 비교
2. **H2O [NeurIPS '23]**
 - 누적 attention 기반 KV cache pruning 기법
3. **SnapKV [NeurIPS '24]**
 - Observation window 기반 KV cache pruning 기법
4. **Vanilla Speculative Decoding**
 - Qwen2.5-3B/Qwen2.5-0.5B 활용 Vanilla SD 구현 실습
5. **EAGLE-style tree (어텐션 마스크 / 확장 / 검증)**
 - Tree mask, Tree verify, Accept/reject 실습
6. **DFlash Parallel Block Drafting [ICML '26]**
 - Diffusion LLM을 통한 한번의 forward로 전체 drafting 실습

Why Decoding is Memory-Bound

3

54

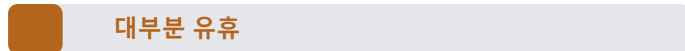
- **Decoding은 한 번에 토큰 하나만 생성**
 - 토큰 하나를 만들기 위해 weight 행렬 전체를 읽어야 함
 - Batch size가 1이면 모든 matrix multiply가 GEMV가 됨
- **Arithmetic intensity가 하드웨어 비율보다 훨씬 낮음**
 - $\text{FLOP/Byte of GEMV} \ll (\text{peak FLOPS}) / (\text{memory bandwidth})$
 - Compute units idle
 - memory bandwidth is the bottleneck

토큰 1개 생성 시

Memory bandwidth



Compute units



해결 방안

- 메모리 읽는 양을 줄인다 → KV Cache Pruning
- 읽어온 김에 연산을 더 한다 → Speculative Decoding

KV Cache Pruning, Speculative Decoding을 통해 모델 크기는 그대로 두고, 추론시간의 메모리와 지연을 줄인다

KV Cache Pruning

- 긴 문맥에서 선형으로 커지는 KV Cache를 중요한 토큰의 Key/Value만 예산 안에 남김
- StreamingLLM · H2O · SnapKV를 동일한 scoring 인터페이스로 구현
- 정확도(passkey retrieval) vs 캐시 메모리를 비교

Speculative Decoding

- 작은 draft 모델이 여러 토큰을 미리 제안
- target 모델이 한 번의 forward로 검증
- chain → tree → parallel block으로 drafting 방식을 확장
- 출력은 AR decoding과 토큰 단위로 동일

두 기법 모두 *memory-bound인 decoding 단계를 타겟하여 최적화를 진행함*

PART 1

KV Cache의 구조와 메모리 병목

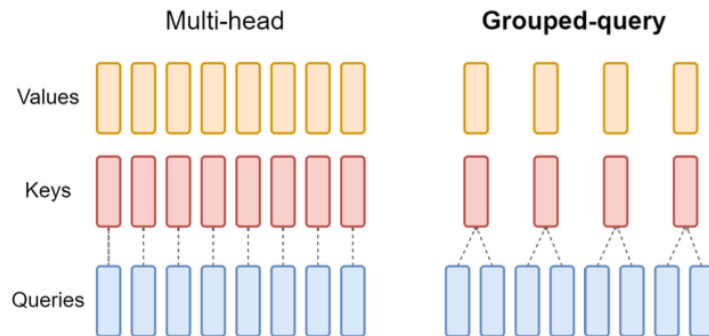
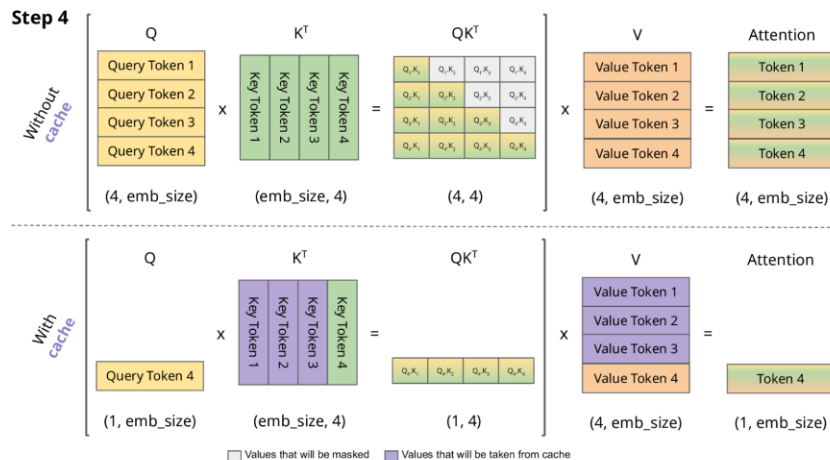
왜 decoding이 memory-bound이고, 캐시가 얼마나 커지는가

What is the KV Cache?

6

54

- 이미 계산한 Key/Value를 저장해 재계산을
없앤다
 - 캐시가 없으면 매 step마다 전체
시퀀스의 K/V를 다시 계산
 - 캐시가 있으면 새 토큰의 K/V만 계산하고
이어 붙임
- 캐시 크기가 문맥 길이에 정비례해 커짐
 - Layer마다, KV head마다 따로 저장
 - GQA는 KV head 수를 줄여 캐시를 크게
절감
 - Ex) Qwen2.5-1.5B
12 query heads but only 2 KV
heads → 6x smaller cache



[실습 1] KV Cache 메모리 계산

7

54

```
def kv_cache_bytes(config, seq_len, batch_size=1, dtype_bytes=2):
    """
    KV Cache가 차지하는 총 바이트 수를 계산합니다.

    Args:
        config      : transformers의 PretrainedConfig
        seq_len     : 문맥 길이 (토큰 수)
        batch_size  : 배치 크기
        dtype_bytes : 원소 하나당 바이트 (FP32 -> 4, FP16/BF16 -> 2)

    Returns:
        int: 총 바이트 수
    """
    nconfig = model.config
    n_layers = config.num_hidden_layers
    n_q_heads = config.num_attention_heads
    n_kv_heads = config.num_key_value_heads
    head_dim = getattr(config, "head_dim", config.hidden_size // config.num_attention_heads)

    # ===== YOUR CODE STARTS HERE =====

    total_bytes =

    # ===== YOUR CODE ENDS HERE =====

    return total_bytes
```

[실습 1] Answer

8

54

```
def kv_cache_bytes(config, seq_len, batch_size=1, dtype_bytes=2):
    """
    KV Cache가 차지하는 총 바이트 수를 계산합니다.

    Args:
        config      : transformers의 PretrainedConfig
        seq_len     : 문맥 길이 (토큰 수)
        batch_size  : 배치 크기
        dtype_bytes : 원소 하나당 바이트 (FP32 -> 4, FP16/BF16 -> 2)

    Returns:
        int: 총 바이트 수
    """
    nconfig = model.config
    n_layers = config.num_hidden_layers
    n_q_heads = config.num_attention_heads
    n_kv_heads = config.num_key_value_heads
    head_dim = getattr(config, "head_dim", config.hidden_size // config.num_attention_heads)

    # ===== YOUR CODE STARTS HERE =====

    total_bytes = 2 * batch_size * n_layers * n_kv_heads * seq_len * head_dim * dtype_bytes

    # ===== YOUR CODE ENDS HERE =====

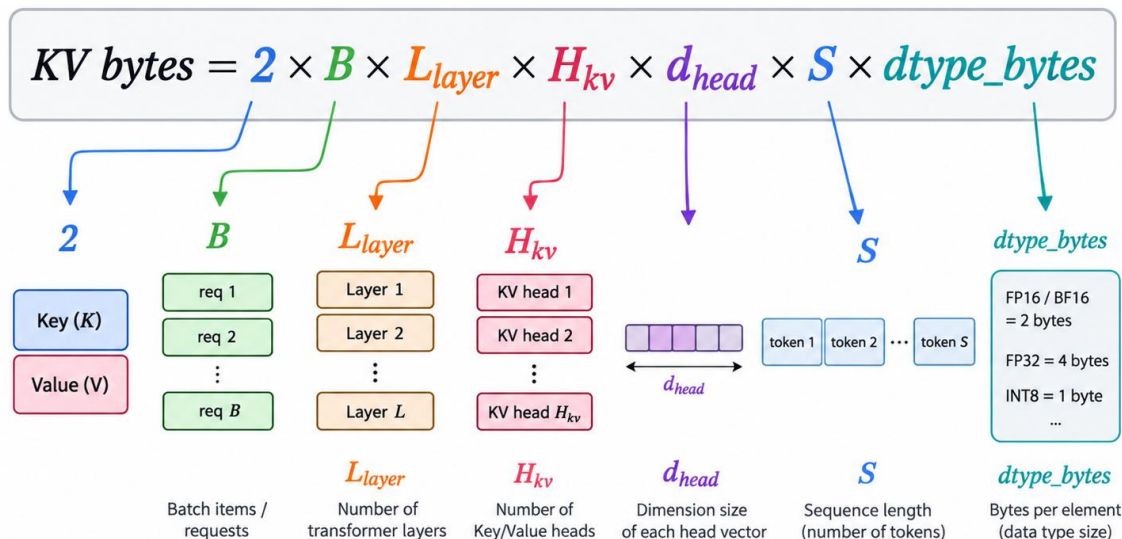
    return total_bytes
```

[실습 1] KV Cache 메모리 계산

9

54

- Implement `kv_cache_bytes(config, seq_len, batch_size, dtype_bytes)`
 - $KV\ bytes = 2 \times B \times L_{layer} \times H_{kv} \times d_{head} \times S \times dtype_bytes$
 - The leading 2 counts Key and Value
 - H_{kv} is the number of KEY/VALUE heads, not query heads



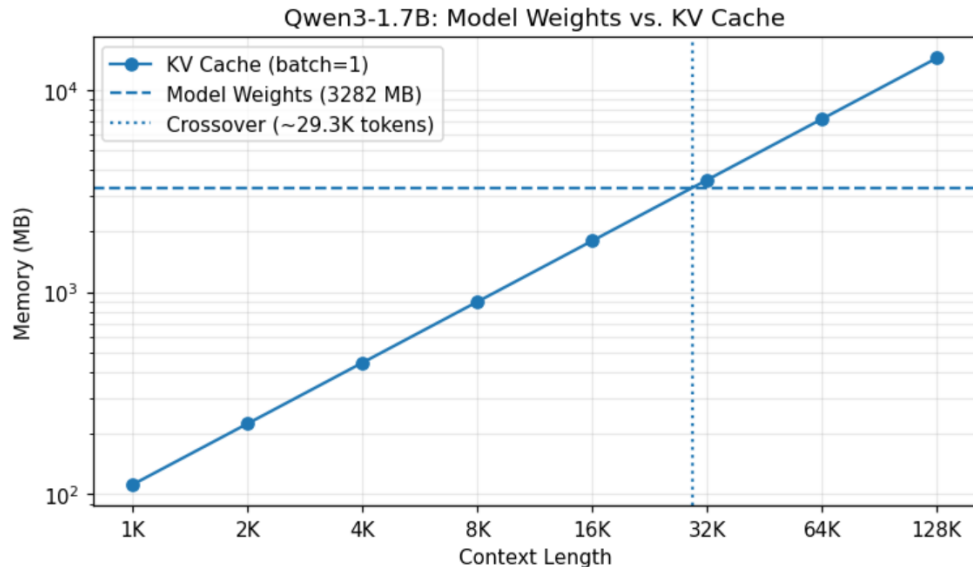
[실습 1] KV Cache 메모리 계산

10

54

Qwen3-1.7B · FP16 에서의 관찰

- 문맥 길이에 정확히 선형으로 증가
- 어느 지점부터 KV Cache가 model weight보다 커짐
- 그 crossover 이후에는 weight 양자화보다 캐시 읽기가 더 큰 병목
- Batch size가 클수록, 문맥 길이가 길수록, KV Cache 문제가 커짐



Model weights : 3281.7 MB
KV cache / token : 112.0 KB
Crossover : ~29.3K tokens

PART 2

KV Cache Pruning

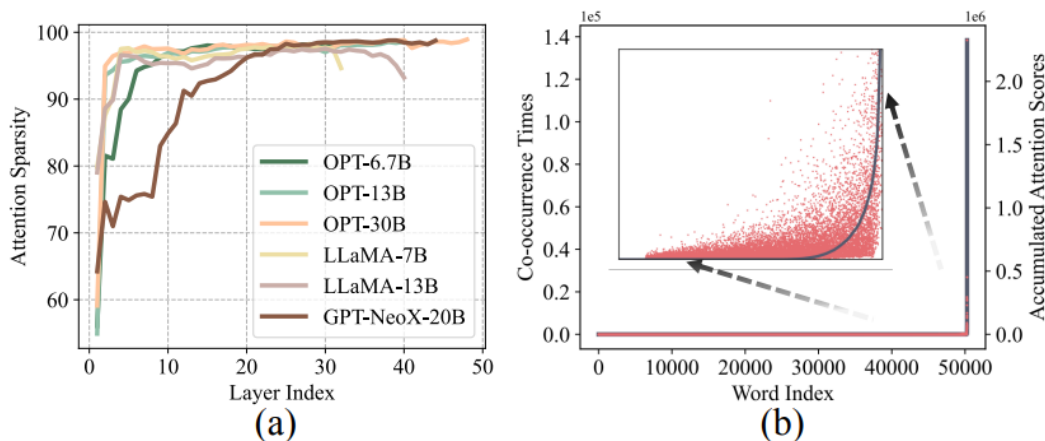
무엇을 남길 것인가 — StreamingLLM · H2O · SnapKV

Attention is Sparse

12

54

- **Attention은 매우 희소하고, 토큰 중요도는 heavy-tailed**
 - 각 decoding step에서 극히 일부 과거 토큰만 큰 attention을 받는다
 - 누적 attention score는 power-law 분포를 보인다
- **This creates an opportunity for KV-cache compression**
 - 많은 KV entry를 버려도 품질을 유지할 수 있다
 - The key challenge : **which KV entries should be retained?**



KV Cache Pruning: Two Design Axes

13

54

Scoring : 무엇이 중요한가

- 위치만 사용 → StreamingLLM
- 누적 Attention Score → H2O
- 누적 Attention Score + observation window → SnapKV

Budget : 얼마나 남길 것인가

- layer 균등 → H2O, SnapKV
- layer별 피라미드 → PyramidKV
- head별 적응적 배분 → Ada-KV, HeadKV

StreamingLLM: Attention Sinks [ICLR'24]

14

54

- **A naive sliding window collapses**
 - 맨 앞 토큰 몇 개가 윈도우를 벗어나는 순간 perplexity가 폭증
- **Cause: attention sinks**
 - softmax는 가중치 합을 1로 맞춰야 하므로 남는 가중치를 어딘가에 쏟는다
 - 맨 앞 토큰들이 그 잉여를 흡수하는 sink 역할을 한다
 - 초기 4개 토큰만 유지해도 잃었던 품질이 거의 회복된다
- **가장 값싼 정책**
 - Keeps the first n_{sink} tokens + a recent window, ignoring attention entirely

유지되는 KV



한계

- 중간 문맥을 통째로 버린다
- passkey가 앞이나 중간에 있으면 실패
- "최근 문맥만 중요한" 스트리밍 대화에 적합

```
def streaming_score(attn_kv, layer, budget, n_sink=4):  
    B, H, K, _ = layer.keys.shape  
  
    # 최근 token일수록 높은 점수: 0, 1, 2, ..., K-1  
    scores = torch.arange(K, device=DEVICE).float()  
    scores = scores.repeat(B, H, 1)  
  
    # 처음 n_sink개 token은 무조건 유지  
    scores[:, :, :n_sink] = float("inf")  
    return scores
```

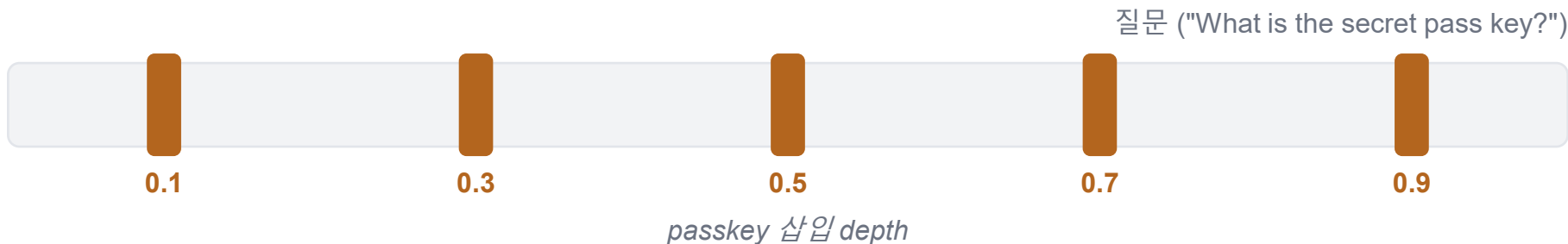
- **attn_kv를 아예 사용하지 않는다**
 - attention 계산이 필요 없어 가장 값싼 정책
- **위치 자체가 score**
 - topk를 하면 자연스럽게 최근 window가 선택된다
 - sink 토큰은 inf로 강제 보존
- **이후 두 방법의 baseline 역할**

Evaluation: Passkey Retrieval

16

54

- 긴 filler 텍스트 안에 5자리 key를 숨기고 질문
 - depth 0.1 / 0.3 / 0.5 / 0.7 / 0.9 로 삽입 위치를 제어
 - 5개 위치 중 몇 개를 맞추는지를 통해 정확도를 계산



```
depth=0.1 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.3 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.5 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.7 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.9 | answer=17412 | 0 | The secret pass key is **17412**
```

Full KV Cache에서의 결과 예시

StreamingLLM 결과

17

54

depth=0.1		answer=17412		X		The secret pass key is **12345**
depth=0.3		answer=17412		X		The secret pass key is **12345**
depth=0.5		answer=17412		X		The secret pass key is **12345**
depth=0.7		answer=17412		X		The secret pass key is **12345**
depth=0.9		answer=17412		0		The secret pass key is **17412**

유지되는 KV

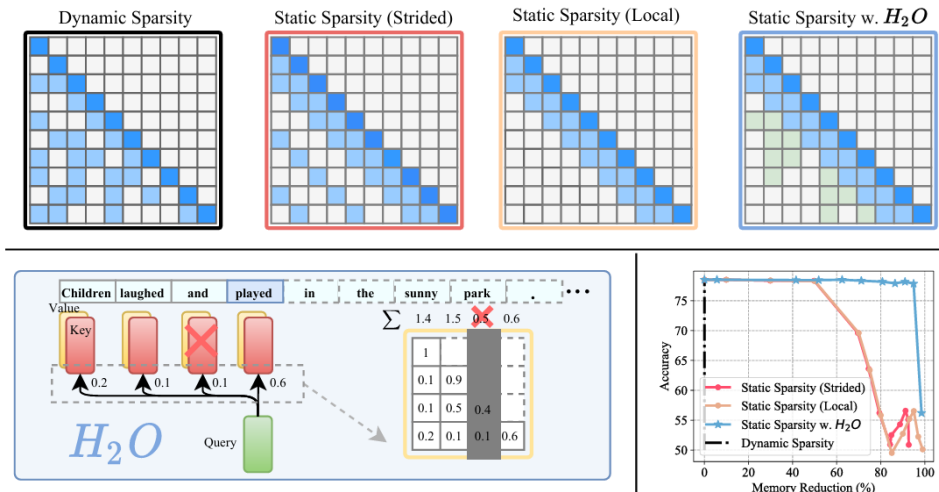


중간에 Evicted 되는 부분에
PassKey가 있을 경우 맞힐 수 없음

H2O: Heavy-Hitter Oracle [NeurIPS '23]

18

54



- **Keep the tokens that accumulate the most attention over all queries**
 - $score(j) = \sum A[i][j]$
 - Combined with a recent window, as in every eviction method
- **Known weakness: recency bias from the causal mask**
 - Token j is only visible to $N-j$ queries, so a plain sum favors early tokens

① 추가

새 토큰의 K/V를 캐시에 넣는다

② 갱신

각 토큰의 누적 attention score를 더한다

③ 축출

예산을 넘으면 score 최소 토큰을 버린다

- 예산 k는 두 몫으로 나눔
 - heavy hitter가 절반, 최근 토큰(recent window)이 절반
 - recent가 없으면 방금 만든 문맥이 끊김
- 로컬 통계만으로 동작
 - 미래 attention 없이, 현재 캐시에 남은 token의 누적 attention만 추적
 - 축출된 token은 이후 attention 계산 및 importance 갱신에서 제외

캐시 예산 k의 구성

Heavy Hitter

score 기준 선택

Recent Window

위치 기준 고정

본 실습의 단순화

step마다 축출하는 대신, prefill이 끝난 뒤 한 번만 pruning한다 (세 기법 비교를 위해 조건을 통일)

- **Implement H2OPress.score()**
 - Reduce the query axis (dim=-2) of the attention weights
 - Keep the key axis intact
- **Then compare H2O and H2O-norm on passkey retrieval**

2-5. H2O Score

```
#####실습코드 시작#####  
# 전체 query의 attention 누적  
scores = _____  
  
# 최근 recent_size개 token은 항상 보존  
scores[..., -recent_size:] = float("inf")  
#####실습코드 끝#####
```

[실습 2] Answer

21

54

```
def h2o_score(attn_kv, layer, budget):
```

```
    recent_size = budget//2
```

```
    #####실 습 코드 시작#####
```

```
    # 전체 query의 attention 누적
```

```
    scores = attn_kv.sum(dim=-2)
```

```
    # 최근 recent_size개 token은 항상 보존
```

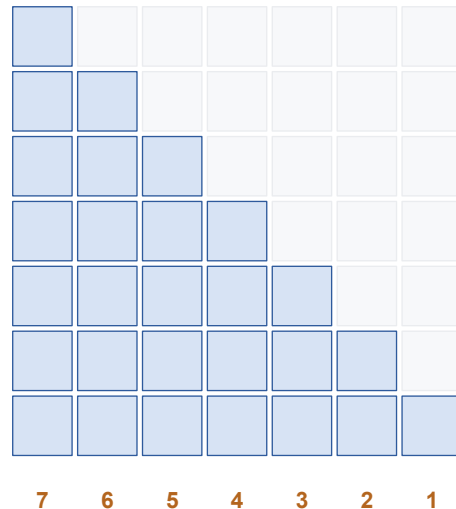
```
    scores[..., -recent_size:] = float("inf")
```

```
    #####실 습 코드 끝#####
```

```
    return scores
```

```
depth=0.1 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.3 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.5 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.7 | answer=17412 | output=The secret pass key is **12345** | X
depth=0.9 | answer=17412 | output=The secret pass key is **12345** | X
```

- causal mask 때문에 앞쪽 토큰일수록 더 많은 query에서 합산된다
 - 위치 j 는 $K-j$ 개의 query에게만 보인다
 - 단순 합산은 앞쪽 토큰에 유리하게 왜곡된다 (원 논문도 지적)
- H2O-norm: 실제로 attend된 query 개수로 나눈다



각 열이 실제로 받은 query 수

```
count = torch.arange(K, 0, -1, device=..., dtype=...)
scores[..., -recent_size:] = float("inf")
return scores / count.view(1, 1, K)
```



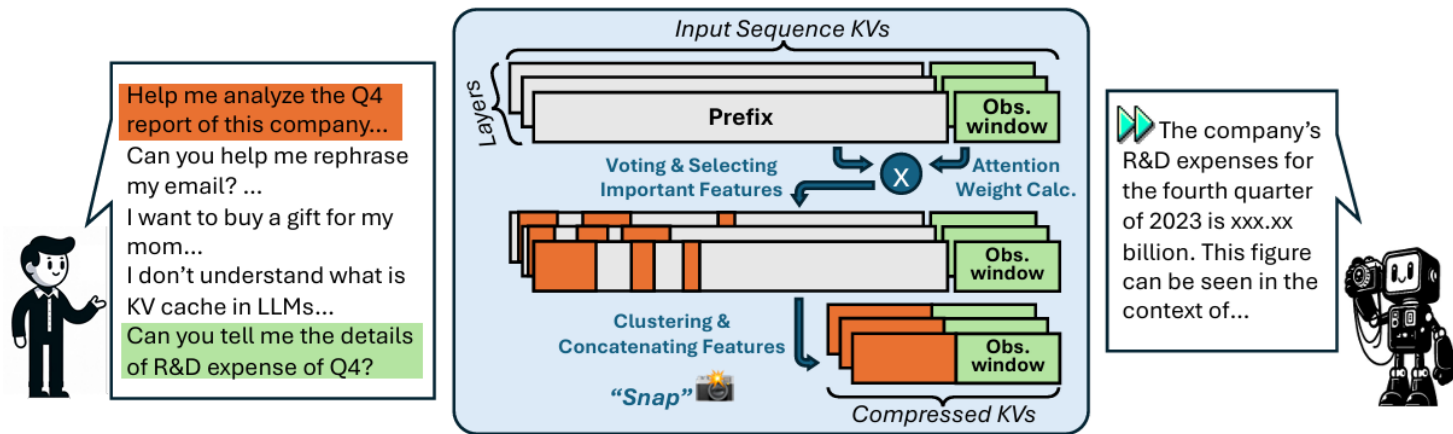
depth=0.1		answer=17412		output=The secret pass key is **17412**		0
depth=0.3		answer=17412		output=The secret pass key is **17412**		0
depth=0.5		answer=17412		output=The secret pass key is **17412**		0
depth=0.7		answer=17412		output=The secret pass key is **17411**		X
depth=0.9		answer=17412		output=The secret pass key is **17412**		0

정규화 유무를 같은 budget에서 비교

SnapKV: Observation Window + Pooling [NeurIPS '24]

23

54



- **Idea 1 - Observation window**
 - What the model will look at during generation is already visible in the attention of the last ~32 query positions of the prompt
 - Using only that window also removes H2O's recency bias
- **Idea 2 - Pooling clustering**
 - Raw top-k picks scattered, isolated tokens: broken fragments of text
 - 1D max pooling over the score makes neighbours survive together, so whole phrases are preserved

SnapKV의 동작

24

54

①

프롬프트를 prefix와 observation window로 나눔

②

window의 query가 prefix에 준 attention을 head별로 합산

③

1D max pooling으로 점수를 클러스터링

④

head마다 top-k를 골라 prefix KV를 압축

▪ Observation window — 전체 query가 아니라 마지막 w개 query만 사용

- 생성 시점에 모델이 볼 곳을 프롬프트 끝부분이 미리 보여준다
- 합산 대상 query 수가 모든 열에서 동일 → H2O의 recency 편향이 사라진다

프롬프트의 분할

prefix (압축 대상)

obs. w

이 query들의 attention으로 채점

▪ Pooling clustering — raw top-k는 흩어진 고립 토큰을 고름



→ 문장 조각만 남음

kernel = 1



→ 주변 문맥이 통째로 살아남

kernel = 7

[실습 3] SnapKV 구현

25

54

- Step 1: aggregate attention over the last w queries only
- Step 2: apply max pooling(kernel_size=k, padding=k//2, stride=1)

2-7. SnapKV Score

```
#####실습코드 시작#####
# TODO 1
# 마지막 w개의 query가 각 KV token에 준 attention을 합산하세요.
# Hint: attn_kv shape = [B, H, Q, K]
scores = _____
...
for j in range(K):
    ...
    # TODO 2
    # Hint: start ~ end 위치 중 최대 score
    pooled_scores[:, :, j] = _____
    ...
#####실습코드 끝#####
```

[실습 3] Answer

26

54

```
def snapkv_score(attn_kv, layer, budget, window=32, kernel=7):
```

```
    B, H, Q, K = attn_kv.shape
```

```
    w = min(window, Q)
```

```
#####실습코드 시작#####
```

```
    # TODO 1
```

```
    # 마지막 w개의 query가 각 KV token에 준 attention을 합산하세요.
```

```
    # Hint: attn_kv shape = [B, H, Q, K]
```

```
    scores = attn_kv[:, :, -w:, :].sum(dim=2)
```

```
    # 각 token 주변 kernel 범위에서 가장 큰 score를 가져옵니다.
```

```
    half = kernel // 2
```

```
    pooled_scores = scores.clone()
```

```
    for j in range(K):
```

```
        start = max(0, j - half)
```

```
        end = min(K, j + half + 1)
```

```
    # TODO 2
```

```
    # Hint: start ~ end 위치 중 최대 score
```

```
    pooled_scores[:, :, j] = scores[:, :, start:end].max(dim=-1).values
```

```
    scores = pooled_scores
```

```
    # Observation window는 항상 유지
```

```
    scores[:, :, -w:] = float("inf")
```

```
#####실습코드 끝#####
```

```
    return scores
```



```
depth=0.1 | answer=17412 | output=The secret pass key is **17412** | 0
```

```
depth=0.3 | answer=17412 | output=The secret pass key is **17412** | 0
```

```
depth=0.5 | answer=17412 | output=The secret pass key is **17412** | 0
```

```
depth=0.7 | answer=17412 | output=The secret pass key is **17412** | 0
```

```
depth=0.9 | answer=17412 | output=The secret pass key is **17412** | 0
```

통합 비교: 정확도 vs KV Cache 메모리

27

54

StreamingLLM

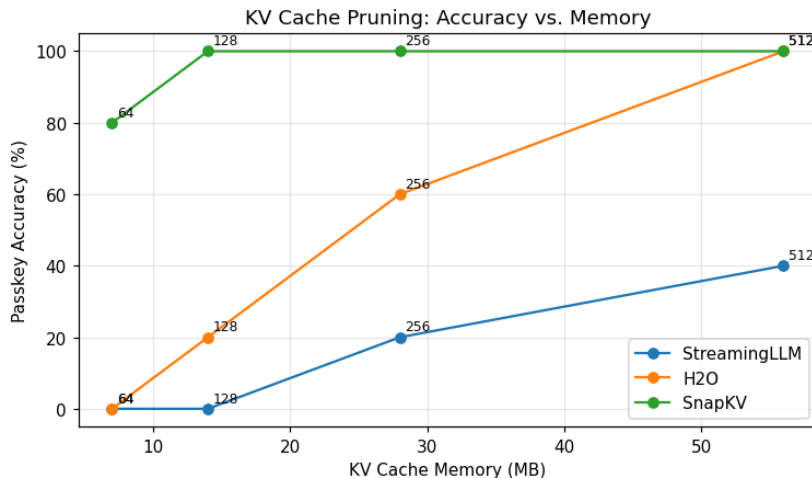
- 위치 기반 (attention 불필요)
- 가장 값쌌음 — prefill attention 없음
- 앞·중간 passkey에서 실패
- 최근 문맥만 중요한 스트리밍에 적합

H2O

- 전체 query 누적 attention
- 중간의 중요 토큰도 포착
- causal mask에 의한 recency 편향
- budget을 줄이면 먼저 무너짐

SnapKV

- 마지막 w개 query + max pooling
- 편향 없음 + 문맥 덩어리 보존
- window 크기·kernel에 민감
- 같은 예산에서 가장 안정적



eviction 계열의 근본 한계

한 번 버린 토큰은 되살릴 수 없다. 프롬프트를 볼 때 중요하지 않던 토큰이 뒤에서 필요해지면 방법이 없다.

다른 KV cache 최적화 연구 방향들

KV 양자화

KIVI
TurboQuant

버리지 않고 비트 수를 줄임

저랭크 분해

Palu

KV를 저차원으로 투영해 저장

Sparse attention

Quest
Block Attention

전체 캐시는 유지하고
일부만 읽는다

예산 배분

PyramidKV
Ada-KV

layer·head별로 budget
을 다르게 설정

PART 3

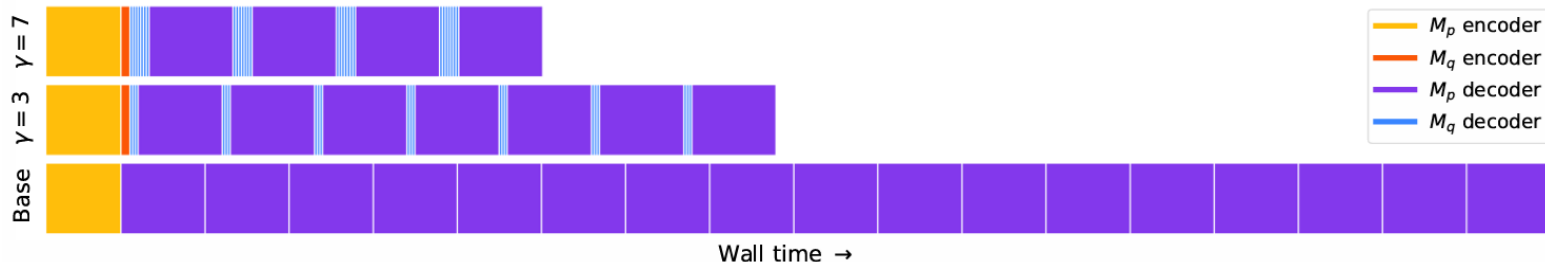
Speculative Decoding 기초

값싼 모델이 제안하고, target이 한 번에 검증한다

Speculative Decoding: Draft and Verify

30

54



- **decoding이 memory-bound이므로, 한 토큰을 만드는 값으로 여러 토큰을 검증할 수 있다**
 - Draft — 작은 모델이 다음 γ 개 토큰을 추측한다
 - Verify — target 모델이 $\gamma+1$ 개 위치를 한 번의 forward로 채점한다
 - Accept — 앞서서부터 일치하는 prefix를 수락하고, target 자신의 다음 토큰 1개를 더 붙인다

Why It Is Lossless

31

54

- **Greedy decoding**
 - A drafted token is accepted only if it equals the target's own argmax
 - The first mismatch is replaced by the target's token
 - The output is token-for-token identical to plain autoregressive decoding
- **Sampling**
 - Rejection sampling preserves the target distribution exactly
- *SD는 다른 압축 기법과 함께 사용 가능*

Quantization / Pruning

- 모델 자체를 바꾼다 → lossy
- 정확도 하락을 감수한다

Speculative Decoding

- 모델을 바꾸지 않는다 → lossless
- 속도만 얻고 품질은 그대로

$$\text{speedup} \approx \tau / (1 + c \cdot \gamma)$$

T acceptance length

라운드당 평균 수락 토큰 수

C drafter cost ratio

target 대비 drafter 1회 forward 비용

Y draft length

라운드당 draft 토큰 수

- τ 만 키우면 되는 것이 아님 $\rightarrow$ drafting 비용이 분모에 있다
 - draft를 길게 뽑으면 τ 는 오르지만 $c \cdot \gamma$ 도 함께 오른다
 - 이후 나오는 모든 기법은 이 trade-off에 대한 서로 다른 답이다

[실습 4] Vanilla Speculative Decoding 구현

33

54

3-2. Draft Generation

```
def draft_tokens(model, x, cache, gamma):
    tokens = []
    # greedy_step()으로 draft token을 gamma개 순차 생성
    for _ in range(gamma):
        x, cache = model.generate(x, cache)
        tokens.append(x)
    return torch.cat(tokens, dim=-1), cache
```

3-3. Target Verification

```
def verify_draft(target, x, drafted, cache):
    # 마지막 committed token + draft token을 하나의 sequence로
    verify_ids = torch.cat([x, drafted])
    result = target(verify_ids, past_key_values=cache,
                    use_cache=True)
    preds = result.logits[0].argmax(-1)
    return preds, result.past_key_values
```

- **draft는 순차 (gamma 회 forward)**

- 여기가 SD의 비용 항 $c \cdot \gamma$

- **verify는 단 1회 forward**

- $\gamma+1$ 개 위치를 동시에 채점
- 마지막 확정 토큰을 앞에 붙이는 이유
- preds[0]이 d1과 비교할 대상이 되기 때문

[실습 4] Vanilla Speculative Decoding 구현

34

54

3-4. Accept / Reject

```
def accept_draft(drafted, preds, gamma):
    n = 0
    for i in range(gamma):
        if _____:
            n += 1
        else:
            break
    # 첫 mismatch에서 정지

    # accept된 token 뒤에 target의 prediction 하나를 추가
    new_tokens = torch.cat([drafted[:, :n], preds[n].view(1, 1)], dim=1)
    return new_tokens, n
```

- Draft와 Target prediction을 앞에서부터 비교
- 첫 mismatch에서 정지

[실습 4] Answer

35

54

```
# =====  
# 3-2. Draft Generation  
# =====
```

```
@torch.inference_mode()  
def draft_tokens(model, x, cache, gamma):  
    tokens = []  
  
    ##### 실습 시작 #####  
  
    # TODO: 다음 draft token을 생성하세요.  
    # HINT: greedy_step()을 사용해 draft token을 gamma개 순차적으로 생성하세요.  
    for _ in range(gamma):  
        x, cache = greedy_step(model, x, cache)  
        tokens.append(x)  
  
    ##### 실습 끝 #####  
  
    return torch.cat(tokens, dim=-1), cache
```

```
@torch.inference_mode()  
def verify_draft(target, x, drafted, cache):
```

```
    ##### 실습 시작 #####
```

```
    # TODO : 마지막 committed token과 draft token들을 하나의 sequence로 연결하세요.  
    # HINT: torch.cat()을 사용해 마지막 committed token과 draft token들을 sequence 방향으로 연결하세요.  
    verify_ids = torch.cat([x, drafted], dim=-1)
```

```
    result = target(  
        verify_ids,  
        past_key_values=cache,  
        use_cache=True  
    )
```

```
    preds = result.logits[0].argmax(-1)
```

```
    ##### 실습 끝 #####
```

```
    return preds, result.past_key_values
```

[실습 4] Answer

36

54

```
def accept_draft(drafted, preds, gamma):

    ##### 실습 시작 #####

    #TODO: Draft와 Target prediction을 앞에서부터 비교하고 첫 mismatch에서 멈추세요.
    n = 0
    for i in range(gamma):
        if drafted[0, i] == preds[i]:
            n += 1
        else:
            break

    # HINT: accept된 draft token 뒤에 Target의 prediction 하나를 추가하세요.
    new_tokens = torch.cat([drafted[:, :n], preds[n].view(1, 1)], dim=1)

    ##### 실습 끝 #####

    return new_tokens, n
```

Gamma Sweep

37

54

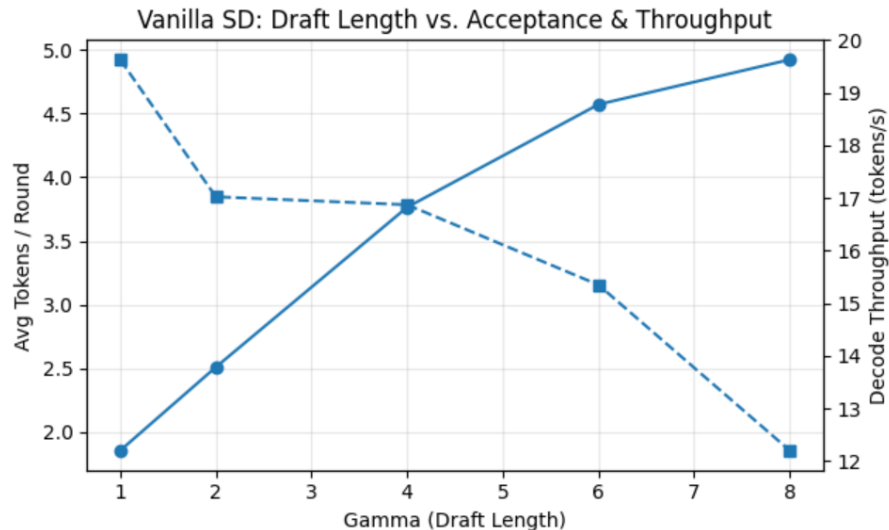
Gamma= 1 | Avg tokens/round=1.86 | TPS=19.6

Gamma= 2 | Avg tokens/round=2.51 | TPS=17.0

Gamma= 4 | Avg tokens/round=3.76 | TPS=16.9

Gamma= 6 | Avg tokens/round=4.57 | TPS=15.4

Gamma= 8 | Avg tokens/round=4.92 | TPS=12.2



- γ 를 키우면 τ 는 계속 오름
 - 한 라운드에서 더 멀리 전진할 수 있기 때문
- TPS는 어느 지점에서 꺾임
 - drafter forward도 γ 번 늘어나기 때문 (분모의 $c \cdot \gamma$)
 - 틀린 뒤의 draft는 전부 버려지므로 낭비가 커진다
- 하드웨어의 특성과 테스트의 특성에 맞춰서 draft length 결정 필수

PART 4

EAGLE 계열: Tree Drafting

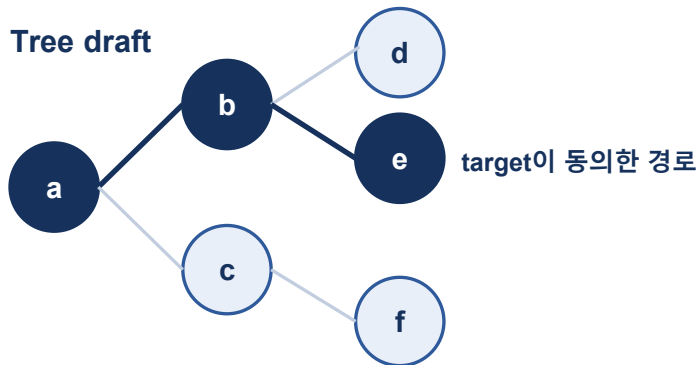
하나의 사슬 대신 여러 후보 경로를 동시에 검증한다

- **A chain draft bets everything on one path**
 - 중간에 한 번 틀리면 그 뒤 토큰은 전부 폐기
- **A tree draft hedges**
 - 여러 후보 경로를 같은 target forward에서 함께 검증
 - target이 동의하는 가지를 고름

Chain draft



Tree draft



한 번의 target forward로 모든 노드를 검증

The EAGLE Family

40

54

drafting 대상

트리

특징

EAGLE-1

feature (hidden state) 공간

정적 트리

token 대신 feature를 예측하고,
shifted token으로 불확실성을 보정

EAGLE-2

feature

동적 트리

drafter의 confidence에 따라
문맥마다 다른 트리를 만든다

EAGLE-3

다층 fused feature → token 직접 예측

동적 트리

feature 예측 제약을 제거,
학습 데이터 스케일링에 잘 반응

본 실습은 AngelSlim/Qwen3-4B_eagle3 체크포인트를 사용하고, 트리 구성·마스크·검증 로직을 직접 구현한다.

Tree Attention Mask and Position IDs

41

54

- Flatten the tree into one sequence, then restrict what each node can see
 - ancestor-only mask: prefix + 자기 조상 + 자기 자신
 - 형제·사촌 노드는 서로에게 보이면 안 된다
- Positions follow tree DEPTH, not sequence order
 - $\text{position_id}(\text{node}) = \text{prefix_len} - 1 + \text{depth}(\text{node})$
 - 형제 노드는 같은 position id를 공유한다

flatten: root a $\rightarrow$ b, c $\rightarrow$ d, e

	a	b	c	d	e	depth
a						0
b						1
c						1
d						2
e						2

행 = query 노드, 열 = 볼 수 있는 노드
(b와 c는 서로 보이지 않는다)

[실습 5-A] Tree Mask & Position IDs

42

54

Answer

```
def build_tree_mask_and_positions(mask_index_list, total_tokens,
device=None):
    # 자기 자신은 항상 볼 수 있음 + 0번(anchor)은 모두가 볼 수 있음
    tree_mask = torch.eye(total_tokens + 1, dtype=torch.bool,
device=device)
    tree_mask[:, 0] = True

    # child는 parent가 보는 node를 그대로 상속한다
    for child, parent in enumerate(mask_index_list, start=1):
        _____

    # 보이는 node 수 = 트리 depth
    tree_position_ids = tree_mask.sum(dim=1).long() - 1
    tree_mask = tree_mask.float()[None, None]
    return tree_mask, tree_position_ids
```

- 부모가 자식보다 먼저 오도록 정렬되어 있음
 - 그래서 한 번의 순회로 조상 집합이 완성된다
- |= 는 OR 결과를 왼쪽에 저장
 - 조상의 가시 집합을 그대로 상속 받는 연산
- depth를 따로 계산할 필요가 없다
 - 보이는 노드 수가 곧 깊이

[실습 5-A] Answer

43

54

```
def build_tree_mask_and_positions(
    mask_index_list,
    total_tokens,
    device=None
):
    # 자기 자신은 항상 볼 수 있음
    tree_mask = torch.eye(
        total_tokens + 1,
        dtype=torch.bool,
        device=device
    )
    tree_mask[:, 0] = True

    ##### 실습 시작 #####

    # ToDo: child가 parent가 볼 수 있는 node들을 그대로 볼 수 있게 하세요.
    # |= 는 OR 결과를 왼쪽에 저장하는 연산입니다.
    for child, parent in enumerate(mask_index_list, start=1):
        tree_mask[child] |= tree_mask[parent]

    ##### 실습 끝 #####

    # 보이는 node 수를 이용해 Tree depth 계산
    tree_position_ids = tree_mask.sum(dim=1).long() - 1
    tree_mask = tree_mask.float()[None, None]
    return tree_mask, tree_position_ids

# 구현을 실제 EAGLE에 연결
student_tree.build_tree_mask_and_positions = build_tree_mask_and_positions
```

```
tensor([[1, 0, 0, 0],
        [1, 1, 0, 0],
        [1, 0, 1, 0],
        [1, 1, 0, 1]], dtype=torch.int32)
```

Position IDs: [0, 1, 1, 2]

[실습 5-B] Tree Verification & Acceptance

44

54

4-4. Tree Verification

```
##### 실습 시작 #####

# To Do: Tree candidate 전체를 Target model에 한 번에 입력하세요.
outputs, tree_logits, hidden_state = model( _____ )

##### 실습 끝 #####
```

4-5. Tree Acceptance

```
##### 실습 시작 #####

# To Do: 가장 길게 일치한 branch와 그 accept length를 선택하세요.
best_candidate = _____
accept_length = _____

##### 실습 끝 #####
```

[실습 5-B] Answer

45

54

```
# =====  
# 4-4. [실습 2] Tree Verification  
# =====
```

```
import eagle_model.ea_model as ea_module
```

```
def tree_decoding(  
    model,  
    tree_candidates,  
    past_key_values,  
    tree_position_ids,  
    input_ids,  
    retrieve_indices,  
):  
    position_ids = tree_position_ids + input_ids.shape[1]  
    if position_ids.dim() == 1:  
        position_ids = position_ids.unsqueeze(0)
```

```
##### 실습 시작 #####
```

```
# To Do: Tree candidate 전체를 Target model에 한 번에 입력하세요.  
outputs, tree_logits, hidden_state = model(  
    tree_candidates,  
    output_orig=True,  
    past_key_values=past_key_values,  
    position_ids=position_ids,  
)
```

```
##### 실습 끝 #####  
return logits, hidden_state, outputs
```

```
# =====  
# 4-5. [실습 3] Tree Acceptance  
# =====
```

```
def evaluate_posterior(  
    logits,  
    candidates,  
    logits_processor,  
):
```

```
# 본 실습에서는 Greedy Decoding만 사용  
assert logits_processor is None
```

```
# 각 branch에서 Draft와 Target prediction 비교  
matches = (  
    candidates[:, 1:].to(logits.device)  
    == logits[:, :-1].argmax(dim=-1)  
) .int()
```

```
# 첫 mismatch 전까지 연속으로 맞은 token 수  
accept_lengths = torch.cumprod(  
    matches,  
    dim=1  
) .sum(dim=1)
```

```
##### 실습 시작 #####
```

```
# To Do: 가장 길게 일치한 branch와 그 accept length를 선택하세요.  
best_candidate = accept_lengths.argmax()  
accept_length = accept_lengths[best_candidate]
```

```
##### 실습 끝 #####  
return best_candidate, accept_length, sample_p
```

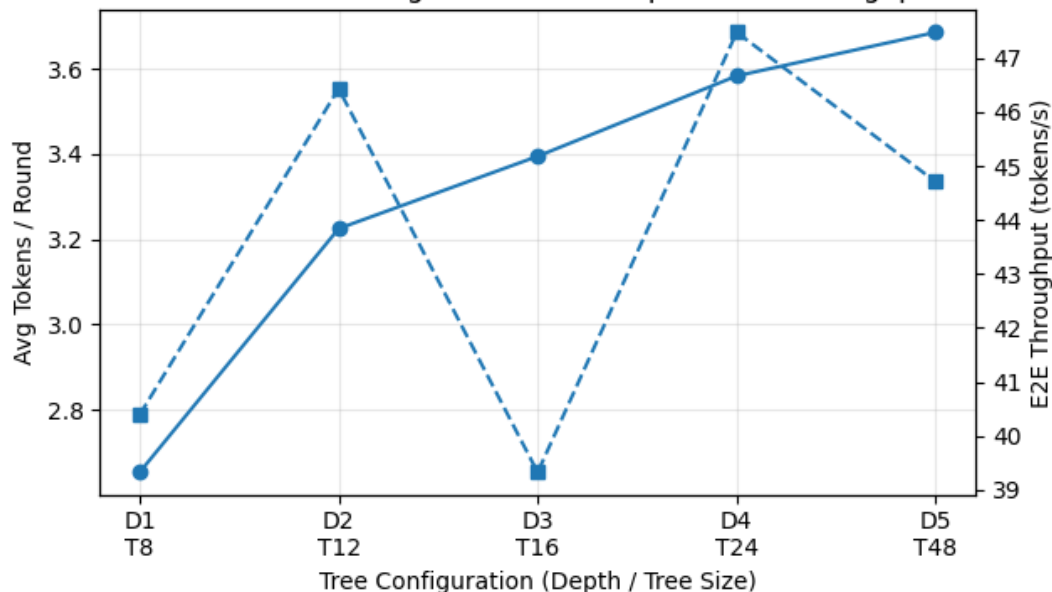
[실습 5-A,B] 결과

46

54

Depth=1 | Tree Size= 8 | Avg Tokens/Round=2.65 | TPS=40.4
Depth=2 | Tree Size=12 | Avg Tokens/Round=3.23 | TPS=46.4
Depth=3 | Tree Size=16 | Avg Tokens/Round=3.39 | TPS=39.3
Depth=4 | Tree Size=24 | Avg Tokens/Round=3.58 | TPS=47.5
Depth=5 | Tree Size=48 | Avg Tokens/Round=3.69 | TPS=44.7

EAGLE-3: Tree Configuration vs. Acceptance & Throughput



Generated Tokens : 129
Target Rounds : 35
Avg Tokens/Round : 3.69
E2E TPS : 26.5 tok/s

[Output]

Sure! Here's a Python function that reverses a **linked list**

Step 1: Define the Node class

```
```python
class Node:
 def __init__(self, value):
 self.value = value
 self.next = None
```
```

Step 2: Define the function to reverse the linked list

```
```python
def reverse_linked_list(head):
 prev = None
 current = head

 while current:
 next_node = current.next
```

# Depth × Tree Size: Drafting은 여전히 Sequential

47

54

- 트리를 키우면  $\tau$ 는 계속 올라간다
  - 후보 경로가 많아질수록 target이 동의할 확률이 높다
- speedup은 그만큼 따라오지 않는다
  - depth를 1 늘릴 때마다 drafter forward가 한 번씩 더 필요하다
  - EAGLE의 drafting은 여전히 autoregressive
  - 이 sequential 비용이 속도 식의 분모를 키운다

depth = 3 인 트리 한 라운드



## PART 5

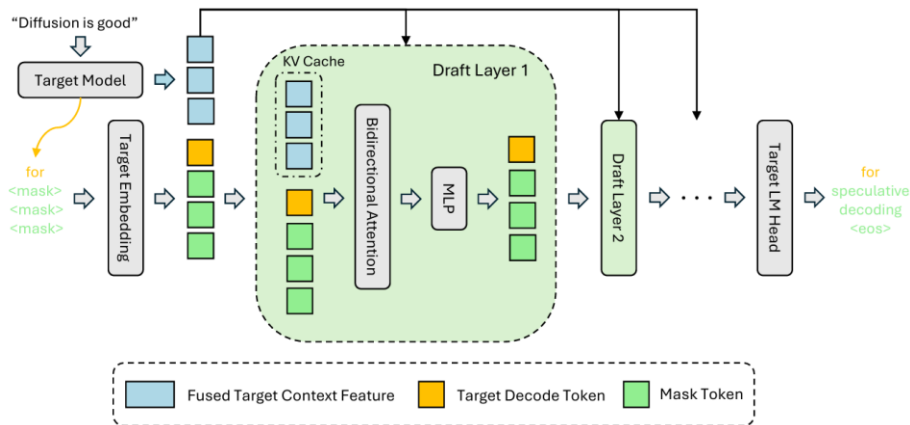
# DFlash: Parallel Block Drafting

drafting의 sequential 비용을 없앤다 · ICML 2026

# DFlash: Block Diffusion Drafting

49

54



- **autoregressive drafter를 block diffusion drafter로 교체한다**
  - 미래 블록을 [MASK] 토큰으로 초기화한다
  - 한 번의 denoising forward로 블록 전체의 분포를 동시에 예측
  - target hidden state를 context feature로 drafter KV에 주입
  - drafter의 logits은 target의 LM head를 통과시킨다
- **Why a diffusion LM works here**
  - 단독 생성 품질은 AR보다 약함
  - verification이 품질을 보장하므로 필요한 것은 병렬성뿐
- **Reported: 6x+ lossless speedup, up to 2.5x over EAGLE-3**

# 한 라운드 비교: Chain / Tree / Block

50

54

## Vanilla SD

drafter forward  $\times \gamma$



Target forward  $\times 1$  (verify)

## EAGLE

drafter forward  $\times$  depth



Target forward  $\times 1$  (verify)

## DFlash

drafter forward  $\times 1$



블록 전체를 동시에

Target forward  $\times 1$  (verify)

세 방식 모두 target 검증은 라운드당 1회. 차이는 drafting에 드는 sequential forward 수

# [실습 6] Parallel Block Drafting

51

54

## 5-1. Parallel Block Drafting

```
실습 시작

To Do: 마지막 num_draft개의 hidden state만 선택하세요.
draft_hidden = _____

To Do: model.compute_logits()을 사용해 모든 위치의 vocabulary logits을 계산하세요.
draft_logits = _____

To Do: torch.argmax()를 사용해 모든 위치의 token을 동시에 선택하세요.
draft_tokens = _____

실습 끝
```

- **argmax에 loop가 없음**
  - 블록 내 모든 위치의 토큰이 같은 forward에서 결정된다
  - compute\_logits는 target의 LM head를 사용한다
  - block size를 키워도 drafter forward 수는 그대로 1회

# [실습 6] Answer

52

54

##### 실습 시작 #####

# To Do: 마지막 num\_draft개의 hidden state만 선택하세요.

```
draft_hidden = draft_hidden[:, -num_draft:, :]
```

# To Do: model.compute\_logits()을 사용해 모든 위치의 vocabulary logits을 계산하세요.

```
draft_logits = model.compute_logits(draft_hidden, output_head)
```

# To Do: torch.argmax()를 사용해 모든 위치의 token을 동시에 선택하세요.

```
draft_tokens = torch.argmax(draft_logits, dim=-1)
```

##### 실습 끝 #####

Draft Shape : torch.Size([1, 15])

[Parallel Draft]

```
['!', ' Here', "'s", ' a', ' Python', ' function', ' that', ' revers', 'es', ' a', ' linked', ' list', ' list', '###', "'ll"]
```

# [실습 6] 결과

53

54

Block Size= 2 | Avg Tokens/Round=1.81 | Decode TPS=24.9  
Block Size= 4 | Avg Tokens/Round=3.17 | Decode TPS=49.7  
Block Size= 8 | Avg Tokens/Round=4.54 | Decode TPS=70.7  
Block Size=12 | Avg Tokens/Round=6.68 | Decode TPS=104.5  
Block Size=16 | Avg Tokens/Round=7.06 | Decode TPS=108.4  
Block Size=32 | Avg Tokens/Round=4.70 | Decode TPS=55.5

Block Size : 16  
Generated Tokens : 128  
Target Rounds : 18  
Avg Tokens/Round : 7.06  
Decode TPS : 108.9 tok/s

[Output]

Sure! Here's a Python function that reverses a **linked list**.

### Step 1: Define the Node class

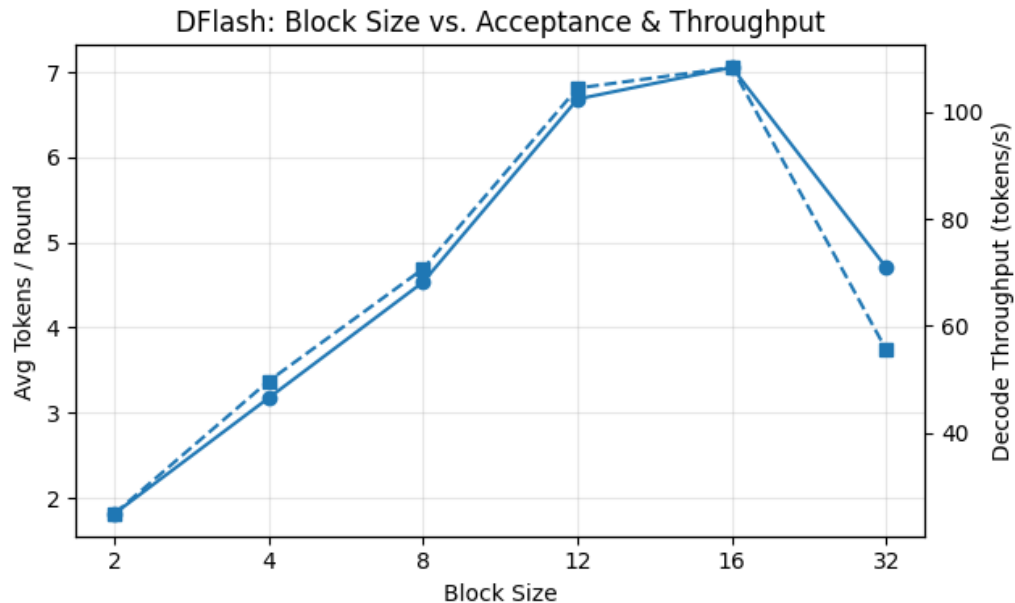
```
```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```
```

### Step 2: Define the function to reverse the linked list

```
```python
def reverse_linked_list(head):
    prev = None
    current = head

    while current:
        next_node = current

```



Q & A

KV Cache Pruning & Speculative Decoding